Object Oriented Programming
Lab Assignment 10

22CS02009
Kumar Snehal

To Run:

1. Please place all the program files in a folder called “Lab_10”, this is because they were created under package name Lab_10, not doing so will give error.
2. To run program, go outside the Lab_10 directory and (# means number)
   a. `javac -cp .:/usr/share/java/mysql-connector-j-9.6.0.jar Lab_10/RestaurantSystem.java`
   b. `java -cp .:/usr/share/java/mysql-connector-j-9.6.0.jar Lab_10.RestaurantSystem`

Problem 1:

The functions `connectToDatabase` and `createTables` create the tables as needed and ensure the foreign key relationship of Customer ID in Orders Table.

```java
    static Statement connectToDatabase() throws Exception {
        String url = "jdbc:mysql://localhost:3306/food_db";
        String username = "KumarSnehal";
        String password = "App@123";
        // Load driver
        Class.forName("com.mysql.cj.jdbc.Driver");
        // Establish connection
        con = DriverManager.getConnection(url, username, password);
        System.out.println("Connected successfully!");
        return con.createStatement();
```

```java
    static void createTables(Statement stmt) throws Exception {
        String createCustomerTable = "CREATE TABLE IF NOT EXISTS Customers (" +
                "customer_id INT PRIMARY KEY AUTO_INCREMENT, " +
                "customer_name VARCHAR(50), " +
                "contact_number INT(10))";
        String createOrdersTable = "CREATE TABLE IF NOT EXISTS Orders (" +
                "order_id INT PRIMARY KEY AUTO_INCREMENT, " +
                "customer_id INT, " +
                "food_item VARCHAR(50), " +
                "quantity INT, " +
                "status BOOLEAN, " +
                "price DECIMAL(7,2), " +
                "FOREIGN KEY (customer_id) REFERENCES Customers(customer_id) ON DELETE CASCADE)";

        stmt.executeUpdate(createCustomerTable);
        stmt.executeUpdate(createOrdersTable);
        System.out.println("Tables created!");
```

Problem 2:

The functions `insert()` and `customerExists()` are used to perform the necessary functions.

```java
    static void insert() {
        System.out.println("Enter Table to Insert (Customers/Orders): ");
        String table = sc.next();
        sc.nextLine();

        if (table.equalsIgnoreCase("Customers")) {
            String insertQuery = "INSERT INTO Customers (customer_name, contact_number) VALUES (?, ?)";

            try (java.sql.PreparedStatement pstmt = con.prepareStatement(insertQuery)) {
                System.out.println("Enter Customer Name: ");
                String name = sc.nextLine();
                System.out.println("Enter Contact Number: ");
                int contact = sc.nextInt();
                sc.nextLine();
                pstmt.setString(1, name);
                pstmt.setInt(2, contact);

                pstmt.executeUpdate();
                System.out.println("Successfully inserted customer!");
            } catch (SQLException e) {
                System.out.println("SQL Exception: " + e.getMessage());
            }

        } else if (table.equalsIgnoreCase("Orders")) {
```

```java
        } else if (table.equalsIgnoreCase("Orders")) {
            String insertQuery = "INSERT INTO Orders (customer_id, food_item, quantity, status, price)
                    + "VALUES (?, ?, ?, ?, ?)";

            try (java.sql.PreparedStatement pstmt = con.prepareStatement(insertQuery)) {
                System.out.println("Enter Customer ID: ");
                int customerId = sc.nextInt();
                sc.nextLine();

                if (!customerExists(customerId)) {
                    System.out.println("Error: Customer ID " + customerId
                            + " does not exist. Please register the customer first.");
                    return;
                }

                System.out.println("Enter Food Item: ");
                String foodItem = sc.nextLine();
                System.out.println("Enter Quantity: ");
                int quantity = sc.nextInt();
                System.out.println("Enter Price: ");
                double price = sc.nextDouble();

                pstmt.setInt(1, customerId);
                pstmt.setString(2, foodItem);
                pstmt.setInt(3, quantity);
                pstmt.setBoolean(4, false);
```

```java
                pstmt.setDouble(5, price);

                pstmt.executeUpdate();
                System.out.println("Successfully inserted order!");
            } catch (SQLException e) {
                System.out.println("SQL Exception: " + e.getMessage());
            }
        } else {
            System.out.println("Invalid table!");
        }
    }

    static boolean customerExists(int customerId) {
        String checkQuery = "SELECT 1 FROM Customers WHERE customer_id = ?";
        try (java.sql.PreparedStatement pstmt = con.prepareStatement(checkQuery)) {
            pstmt.setInt(1, customerId);
            try (ResultSet rs = pstmt.executeQuery()) {
                return rs.next();
            }
        } catch (SQLException e) {
            System.out.println("Database error while verifying customer: " + e.getMessage());
            return false;
```

Problem 3:

The functions `update()` and `retrieve()` are used to perform the necessary functions.

```java
static void update(Statement stmt) {
    System.out.println("Enter Order ID to Update: ");
    int orderId = sc.nextInt();
    sc.nextLine();
    String updateQuery = "UPDATE Orders SET status = true WHERE order_id = " + orderId;
    try {
        stmt.executeUpdate(updateQuery);
        System.out.println("Order status updated to completed!");
    } catch (SQLException e) {
        System.out.println("SQL Exception: " + e.getMessage());
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

```java
static void retrieve(Statement stmt) {
    String retrieveQuery = "SELECT o.order_id, c.customer_name, o.food_item, o.quantity, o.price "
            "FROM Orders o JOIN Customers c ON o.customer_id = c.customer_id and o.status = false";
    try {
        ResultSet rs = stmt.executeQuery(retrieveQuery);
        System.out.println("\n--- Pending Orders ---");
        while (rs.next()) {
            int orderId = rs.getInt("order_id");
            String customerName = rs.getString("customer_name");
            String foodItem = rs.getString("food_item");
            int quantity = rs.getInt("quantity");
            double price = rs.getDouble("price");
            System.out.println(
                    orderId + " | " + customerName + " | " + foodItem + " | " + quantity + " | " +
        }
        System.out.println("====================");
    } catch Loading... ption e) {
        System.out.println("SQL Exception: " + e.getMessage());
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Problem 4:

The function `analyze()` is used to perform the necessary functions.

```java
    static void analyze(Statement stmt) {
        System.out.println("Options:");
        System.out.println("1. Bill for each Order");
        System.out.println("2. Total Spending of each Customer");
        System.out.println("3. Customers who spent more than a certain amount");
        int choice = 0;
        choice = sc.nextInt();
        sc.nextLine();
        String query = "";
        switch (choice) {
            case 1:
                query = "SELECT order_id, price * quantity FROM Orders";
                break;
            case 2:
```

```java
            case 2:
                query = "SELECT c.customer_name, SUM(o.price * o.quantity) AS total_spending "
                        + "FROM Orders o JOIN Customers c ON o.customer_id = c.customer_id "
                        + "GROUP BY c.customer_id";
                break;
            case 3:
                System.out.println("Enter the minimum spending amount: ");
                double minSpending = sc.nextDouble();
                sc.nextLine();
                query = "SELECT c.customer_name, SUM(o.price * o.quantity) AS total_spending "
                        + "FROM Orders o JOIN Customers c ON o.customer_id = c.customer_id "
                        + "GROUP BY c.customer_id "
                        + "HAVING SUM(o.price * o.quantity) > " + minSpending;
                break;
            default:
                System.out.println("Invalid choice!");
                return;
        }
        try {
```

```java
        try {
            ResultSet rs = stmt.executeQuery(query);
            System.out.println("\n--- Analysis Result ---");
            while (rs.next()) {
                if (choice == 1) {
                    int orderId = rs.getInt(1);
                    double bill = rs.getDouble(2);
                    System.out.println("Order ID: " + orderId + " | Bill: " + bill);
                } else {
                    String customerName = rs.getString(1);
                    double totalSpending = rs.getDouble(2);
                    System.out.println("Customer: " + customerName + " | Total Spending: " + totalSpend
                }
            }
            System.out.println("====================");
        } catch (SQLException e) {
            System.out.println("SQL Exception: " + e.getMessage());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
```

Some Additional Functions:

- delete() - On basis of Customer ID or Order ID
- display() - Display both the tables and their contents.

```
kumar-snehal@Kumar-Snehal-PC:~/OOP$ java -cp
Connected successfully!
Tables created!
1. Insert
2. Update
3. Retreive
4. Analyze
5. Delete
6. Display
Else. Exit
1
Enter Table to Insert (Customers/Orders):
Customers
Enter Customer Name:
Snehal
Enter Contact Number:
1234567890
Successfully inserted customer!
```

```
1. Insert
2. Update
3. Retreive
4. Analyze
5. Delete
6. Display
Else. Exit
6

--- Customers ---
1 | Sahil | 999
2 | Snehal | 1234567890
=====================

--- Orders ---
1 | 1 | Panner | 9 | false | 100.0
=====================
```

```
1. Insert
2. Update
3. Retreive
4. Analyze
5. Delete
6. Display
Else. Exit
4
Options:
1. Bill for each Order
2. Total Spending of each Customer
3. Customers who spent more than a certain amount
1

--- Analysis Result ---
Order ID: 1 | Bill: 900.0
====================
```

```
=====================
1. Insert
2. Update
3. Retreive
4. Analyze
5. Delete
6. Display
Else. Exit
7
Exiting...
kumar-snehal@Kumar-Snehal-PC:~/OOP$
```